1. Short answers

1.1

StatelessWidget	StatefulWidget
Static doesn't do a UI update	Can update the UI during run using [setState()]
Simpler and more performant.	More complex, holds the state, deals with dynamic data. Needed for user responsive screens.

1.2 it's responsible for returning the widget tree, the render start point of the screen. And it's called by the framework.

1.3 as said in 1.1 it's calling for a rebuild for the screen - calls the build method, it also marks the widget as dirty and calling it unnecessarily is a bad practice.

1.4 Final : the value can be only assigned once [runtime constant], const is compile time constant.

1.5 Null safety is same as optional in iOS, by adding type? , it can either be null or type, it needs an if check != null or safe assignment ?? 'value'. It forces you to handle nullable situations before run.

2.1 - 1 points

Which widget is commonly used to arrange children vertically?

- A. Row
- ***B. Column***
- C. Stack
- D. Container

2.2 - 1 points

Which widget is commonly used to add padding around another widget?

- ***A. Padding***
- B. Scaffold
- C. Expanded
- D. Navigator

2.3 - 1 points

What is the role of MaterialApp?

• A. It stores all application data permanently

• B. It provides Material Design configuration and app-level setup

• C. It performs HTTP requests

• D. It rebuilds only one widget manually

2.4 - 1 points

Which Dart keyword is used to wait for a Future?

• A. sync

• B. late

• C. await

• D. yield

2.5 - 1 points

What does ListView.builder help with?

• A. Loading all list items at once

• B. Building list items lazily as needed

• C. Creating database tables

• D. Sending HTTP requests

Level 2 - Widgets, State, and Debugging

Difficulty: Easy to Medium | **Score: 15 points**

3.1 - 4 points | Code reading

Consider this Flutter code:
ElevatedButton(
onPressed: rollDice,
child: const Text('Roll dice'),)
When calling a function onPressed : it should be rollDice with no (); or () => rollDice().

4.1 - 4 points | State update

A quiz app stores the selected answer in a variable, but the UI does not update after the user taps an answer.
Give two possible reasons why this could happen.

- Didn't call for a setState() to update the UI. or updated the var outside setState.
- Using StatelessWidget.

5.1 - 4 points | Debugging process

You see this error:
setState() called after dispose()
Explain what this means and give one example of how it could happen.

- dispose() will unmount the UI,so setState shouldn't be called.
- It could happen if setState was called in a timer function.

6.1 - 3 points | Widget extraction

Why can extracting part of a large widget tree into smaller widgets improve code quality? Mention at least two
Benefits.

- Smaller static parts would be defined as [const] so it won't be rebuilt, which will reduce the cost of rebuilding.
- Smaller widgets would be easier to understand, maintain and develop.
- Easier to test and be reused.

Level 3 - Interactivity, Theming, and Responsive UI

Difficulty: Medium | **Score: 15 points**

7.1 - 5 points | Expense tracker scenario

In an expense tracker app, a chart and a list fit well in portrait mode, but overflow in landscape mode on a small
device. Describe at least three techniques you could use to make the UI responsive.

- Using flexible and expanded.
- Layout builder
- MediaQuery to get screen size & orientation:.

8.1 - 4 points | Theming

Explain how ThemeData and Theme.of(context) help avoid duplicated styling across the app.
By defining our main theme in the main.dart app build function, we can use ThemeData to hardcode styles everywhere.
Theme.of(context) is how we use the theme anywhere in the app:

9.1 - 3 points | Adaptive UI

What is the difference between a responsive UI and an adaptive UI?

- Responsive covers different screen sizes, landscape portrait etc same UI different sizes.
- Adaptive covers more range of differences like pc, tablet , phone, it would require more code and even different UI for different devices.

10.1 - 3 points | Interactivity

A user taps a delete icon in an expense list. Describe the flow from user interaction to UI update.

Flow: 1 - user taps delete.
2- onPressed gets called.
3- Expense removed by id inside setState.
4- It triggers a rebuild.
5- List builder renders updated list.

• the callback; delete that expense from the data list. Delete by id.
• the state change; we call setState()
• the rebuild. The page would be rebuilt according to the new list length.

Level 4 - Widget Tree, Element Tree, Render Tree, and Performance

Difficulty: Medium | **Score: 15 points**

11.1 - 5 points | Conceptual understanding

Explain the difference between:
• Widget Tree; the code we write, the widget design the Buttons, Text etc.
• Element Tree: the elements created by the widget, it holds the config and the state of this widget in run.
• Render Tree: what is print to screen, like actual pixels.

To sum them up and put the different responsibilities together , let's say we call for an update, *setState* , the widget Tree would rebuild the code, while the element tree would update the state of it, while the render tree would see if there's a need to update and print something new to the layout and print the new renderObject in question.

12.1 - 4 points | Avoiding unnecessary rebuilds

- Divide the app into smaller widgets.
- always use const when no dynamic function is needed.
- Always use stateless when no need to update.
- Use ListBuilder to build only the needed data to be shown.

13.1 - 3 points | List performance

```
ListView.Builder(
itemCount: items.length,
itemBuilder: (context, index) {
return Text(items[index].title)} );
```

It will load the whole list, it's not efficient and would impact memory. Use builder to only load what it needs to be presented on screen.

14.1 - 3 points | Keys

When can using a Key be useful in Flutter lists or dynamic UIs?
Keys are used to identify the elements to make it easy to [add - delete - move] items or else it would be mixed up.
Same would be useful to identify the widgets if added or removed, so flutter would know which is which. Instead of just knowing by position you know by id, which will help when reorders.

Level 5 - Navigation, App-Wide State, and Animations

Difficulty: Medium to Hard | **Score: 15 points**

15.1 - 4 points | Navigation

A meals app has these screens:
• Categories screen;
• Meals list screen;
• Meal details screen.
Describe how navigation could work when the user selects a category and then selects a meal.

First i would add routes in main

```
    routes: {
    '/': (ctx) => const CategoriesScreen(),
      MealsListScreen.routeName: (ctx) => MealsListScreen(),
      MealsDetailsScreen.routeName: (ctx) => MealsDetailsScreen(),
    },
```

When a user selects a category which I think it would be in a builder, the selected item should be sent from the itemBuilder widget, And when I select category or meal [i would use same approach] . send the data needed to be displayed as an argument.

```
  void selectCategory(BuildContext context) {
  Navigator.of(
    context,
  ).pushNamed(MealsListScreen.routeName, arguments: category);
}
```

How to receive :

```
  final routeArgs = ModalRoute.of(context)?.settings.arguments as Category;


  final routeArgs = ModalRoute.of(context)?.settings.arguments as Meal;
```

Note: sure can use providers, but this is simpler for this case.

16.1 - 3 points | Passing data between screens

- By arguments already explained in 15.1
- By sending to constructors.
- By providers

17.1 - 4 points | App-wide state

A user marks a meal as favorite on the meal details screen. The favorite status must also be visible on the
favorites screen. Where should this state live, and why should it not only be stored inside the details screen?

- It shouldn't be in details, because state dies when screen is popped.
- It should be stored in the provider.
- To be visible across the app as the single source of truth.

18.1 - 4 points | Animations

When would you use an implicit animation such as AnimatedOpacity or AnimatedSwitcher, and when would you
use an explicit animation with an AnimationController?

- I would use implicit animation for simpler transitions as I only need to set the target and durations. Ex: Button pressed.
- Explicit animation gives a full control of the animation which would need you to manually create the AnimationController, you could use it for complex design or if a client design needs to be implemented.

Level 6 - Forms, HTTP, Native Features, and Firebase

Difficulty: Hard | **Score: 25 points**

19.1 - 5 points | Forms

If name after eliminating space was empty. Is null
If quantity after parsing < 1 , or null
If category is null or empty
Ex

```
TextFormField(
  validator: (value) {
    if (value == null || value.trim().isEmpty) {
      return 'Name can't be empty';
    }
    return null;
  },
),
```

20.1 - 4 points | Async form submission

• loading;
Once a user hits submit, I would change the state into loading which according to design may show a spinner or a loading screen.

• success; upon the success of the submission the state would change again returning *data* as a response, that would trigger the success design, dismiss loading, reset the form and then show data or show success dialog.

• failure: upon failing the request it would mostly go to catch case where we change state into failure, we then show an error message.

21.1 - 5 points | HTTP request

Code must be placed in the repo:
If made with an http - Dio

```
final url = Uri.parse('https://example.com/api/grocery-items');

ry {
```

```
    final response = await http.post(
      url,
      headers: {},
      body: item.encodeToJson(),
    );
  } catch (e) {
    throw Exception('Network error: $e');
  }

  final dio = Dio();
  try {
    final response = await dio.post(url.toString(), data: {
      'name': item.name,
      'quantity': item.quantity,
      'category': item.category,
    });
    return response.data;
  } on DioException catch (error) {
    final errorMessage = ErrorMapper.mapErrorToMessage(error);
    throw Exception(errorMessage);
  }
```

Can check for status code, create appFailure class , map errors
// if for http not mapper

```
if (response.statusCode != 201) {
 throw Exception('Failed: ${response.statusCode}');
}
```

```
abstract class AppFailure {
 final String messages;
 AppFailure(this.messages);
}
```

```
class NetworkFailure extends AppFailure {
 NetworkFailure([String messages = "Network Failure"]) : super(messages);
}
```

```
// no auth
class NoAuthFailure extends AppFailure {
 NoAuthFailure([String messages = "No Auth Failure"]) : super(messages);
}
```

```
//server error
```

```
class ServerFailure extends AppFailure {
 ServerFailure([String messages = "Server Failure"]) : super(messages);
}
```

```
//type error
```

```
class TypeErrorFailure extends AppFailure {
 TypeErrorFailure([String messages = "Type Error"]) : super(messages);
}
```

```
class ErrorMaper {
 static AppFailure mapErrorFromDio(dio.DioException ex) {
   switch (ex.type) {
     case dio.DioException.connectionTimeout:
     case dio.DioException.sendTimeout:
     case dio.DioException.receiveTimeout:
       return NetworkFailure();
     case dio.DioException.connectionError:
       return NetworkFailure();
     case dio.DioException.badResponse:
       if (ex.response?.statusCode == 401) {
         return NoAuthFailure();
       } else {
         return ServerFailure();
       }
     default:
       return TypeErrorFailure("Unexpected Error: ${ex.message}");
   }
 }
}
```

22.1 - 4 points | JSON parsing

Given this JSON response:

{
"id": "abc123",
"name": "Milk",
"quantity": 2,
"category": "dairy"
}

Write a simple Dart model class or factory constructor to parse it.

```
class GroceryItem {
 final String id;
 final String name;
 final int quantity;
 final String category;

 const GroceryItem({
   required this.id,
   required this.name,
   required this.quantity,
   required this.category,
 });

factory GroceryItem.fromJson(Map<String, dynamic> json) {
   return GroceryItem(
     id : json['id'],
     name: json['name'],
     quantity: json['quantity'],
     category: json['category'],
   );
 }
}
```

23.1 - 3 points | Native features

A favorite places app allows the user to select a location and take a photo. Name two native device features that
may be involved and explain why permissions may be required.

- Location service.
- Camera service.

Each platform has its own permission for both, especially iOS.
You need to add into info.plist file , and add to AndroidManifest.xml because it's sensitive user data info which uses device hardware.

24.1 - 4 points | Firebase

Explain at a high level what Firebase can be used for in a Flutter app. Mention at least two Firebase services or
Concepts.

- It can be used as a backend for firestore.
- The most famous feature is sending notifications.
- Authentication services.

Practical Task - Mini Feature Design

Difficulty: Hard | **Score: 15 points**

25 - 15 points

Design a small Flutter feature for this scenario:

The app shows a list of favorite places. The user can add a new place with a title, an image, and a location.The place should appear in the list after saving.

You do not need to build the full app. Describe the design. Your answer should include:

1. Main widgets or screens involved.

Screens : HomeScreen [ListScreen] - AddNewPlaceScreen - PlaceDetailsScreen

Widgets : ImageInput - LocationInput

2. State management approach.

Will use state notifier class,

```
final userPlacesProvider =
   StateNotifierProvider<UserPlacesNotifier, List<Place>>(
     (ref) => UserPlacesNotifier(),
   );
```

3. Data model.

Place , PlaceLocation

```
class PlaceLocation {
 final double latitude;
 final double longitude;
 final String? address;

 PlaceLocation({
   required this.latitude,
   required this.longitude,
   this.address,
 });
}
class Place {
 final String id;
 final String title;
 final PlaceLocation location;
 final File? image;

 Place({String? id, required this.title, required this.location, this.image})
   : id = id ?? uuid.v4();
}
```

4. Form validation.

validator: _validateName : must not be empty nor null

_validatePhoto : must not be empty nor null

_validateLocation : must not be empty nor null

5. How image/location data is handled.

After adding all the permission needed , i then add buttons to add location and photos, then execute the code attached to this features,It gets validated then save lang and lat of location but for photo i would save it’s path.

6. How the UI updates after saving.

The ref which we already added the data too , would already be notified of the change.

7. One possible error case and how you would handle it.

If no agree on the permissions I would ask again .

Logical issue: i could even ask user to give me a gallery image or search locations to give co ordinates instead of user location service .

Optional Advanced Section - Production-Style Flutter

Optional bonus section | **Bonus: 20 points**

This section is optional. It is designed to test whether the candidate can move beyond course projects and reason about production-style code.

26 - Bonus - 5 points | Architecture boundaries

In a production Flutter app, explain why HTTP requests and database queries should not be written directly inside
widgets. Where should this logic usually live?

Widgets can be reused and be rebuilt multiple times, which would be heavy to put data operation on it, the ideal way is that we put the request into the repo layer and inject the Dio and the Database into it as a parameter , so it can be easy to test and reused even for different URLs and different database config.

27 - Bonus - 4 points | Repository pattern

What is the responsibility of a repository in a Flutter app? Give an example involving remote data and local persistence.

- To handle data requests, for database and for remote request.
- Ex
  Abstract class IRepo { Future<List<item>> getAll(); }
  Class ItemRepo implements IRepo {
  Final _dio ;
  Final AppDatabas _database;
  ItemRepo(this._*dio, this.*_databse)
- Future<List<item>> getAll() {
- //we can do here a local first call then update the db from remote the return }
  } // full implementation in **28**

28 - Bonus - 4 points | Offline behavior

The app fetches data from a remote API but should still work without internet. Describe a simple offline-first flow.

```
Future<List<Item>> getAll() async {
final localItems = await _database.getAllItems(); // always get local first
if (localItems.isNotEmpty) return localItems;

try { // try to update
  final response = await _dio.get('/items');
  final remoteItems = (response.data as List)
      .map((json) => Item.fromJson(json))
      .toList();

  await _database.insertAll(remoteItems);

  return remoteItems;

} on DioException catch (e) {
  if (localItems.isNotEmpty) return localItems;
  throw Exception('No internet and no cached data'); // or use mapper.
}
}
```

29 - Bonus - 4 points | Error handling

```
class ErrorMaper {
 static AppFailure mapErrorFromDio(dio.DioException ex) {
   switch (ex.type) {
     case dio.DioException.connectionTimeout:
     case dio.DioException.sendTimeout:
     case dio.DioException.receiveTimeout:
       return NetworkFailure();
     case dio.DioException.connectionError:
       return NetworkFailure();
     case dio.DioException.badResponse:
       if (ex.response?.statusCode == 401) {
         return NoAuthFailure();
       } else {
         return ServerFailure();
       }
     default:
       return TypeErrorFailure("Unexpected Error: ${ex.message}");
   }}}
```

Why is it better to handle these cases separately instead of showing one generic error for everything?

Identifying errors make it possible to be fixed, auth error would make me go check my credentials , internet would switch the internet service , while the generic error would just block the whole experience of using this app.

30 - Bonus - 3 points | Local database

At a high level, explain when a local database such as Drift is useful in a Flutter app.
It keeps everything saved and organized also easy to be retrieved, and when no internet connection or no backend at server , it's our main save and offline support.

- Offline data persistence.
- Caching.